

Advanced OS Project 4 Report

Written By: Joshua Sampson

For this project, we had to implement a distributed, replicated, in-memory key-value store, GTStore, using gRPC for communication. Before anything else, it is crucial to understand what a distributed system is and how it differs from its “counterpart”, the centralized system. The motivation behind a “distributed system” is that with centralized storage systems, while simple in design and implementation, suffers from inherent physical limitations such as but not limited to “single point of failure” and bottlenecks for throughput (because all “traffic” goes through one entity). The distributed system addresses these vulnerabilities, which is the architecture implemented in this project. The way data is stored is the main difference, where we decouple the storage logic across multiple networked nodes (Storage Nodes as is called in the project). This allows us to achieve horizontal scalability and fault tolerance. Horizontal scalability is a way to scale a system by adding more machines or resources to the already existing pool of resources rather than just enhancing one single machine and fault tolerance is the ability of the system to continue operating normally despite a failure in one of the systems components (in particular, the storage nodes). However, these fundamental system changes introduce complex challenges not found in centralized programs; some of these challenges include network latency, data replication, data consistency, sharding, and data serialization. Circling back to GT Store, it is designed to handle these listed challenges explicitly. The system architecture revolves around three components: a centralized manager, storage nodes, and the client library.

The GTStore “centralized manager” maintains an up-to-date view of the entire system. When a storage node is initialized and wants to make itself known to the system, it must first “register” through the manager. This is what is called membership management. This behavior is enforced in the Register RPC, inside of manager.cpp, where dynamic node registration is achieved by assigning unique IDs to each new node. As each new node is registered, the manager adds it to the map “nodes” which contains all nodes that have been registered up to the most recent point. It is important to note that this map is the central part of the system, as it also indicates if a node is alive or dead, meaning there is no separate data structure for “alive” nodes, for example. This becomes more obvious as we talk about failure detection. This is implemented within the manager and is designated as a background thread, called “monitor_thread” which executes `check_nodes()` indefinitely, until the system is completely terminated. While the system is running, this background thread, via, `check_nodes()`, sends periodic “Ping” requests (RPC) to each node on the system (stored in “nodes”). Each node is given a one second deadline, and if there is no response within that time, the node is considered dead. This triggers the node to be marked as “dead” and the `handle_node_failure()` function to execute and perform its recovery protocol. Discussion on `handle_node_failure()` will be deferred to a later section where it is more appropriate to discuss its specific implementation details. Finally, manager manages metadata

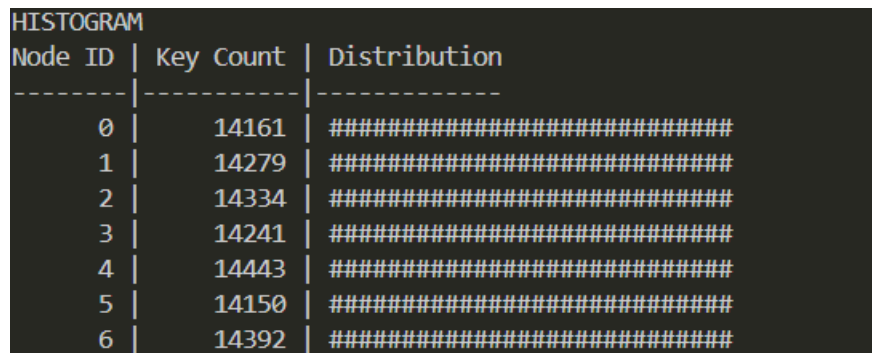
lookup/placement via the `GetNodeForKey` RPC. Here, it calculates the primary partition for a key and sends back to the client in the form of a response the primary node and its $K-1$ replicas. Again, specific implementation details of this will be deferred to later.

The GTStore storage nodes are the second component of the system. These nodes store in-memory key-value pairs for which they serve as either the primary owner and/or a replica. The way data is stored on a node is done via a critical design choice: buckets. As defined in `gtstore.hpp`, the storage node (`GTStoreStorage`) maintains a vector of buckets (`Bucket` struct). Each bucket (index of the “buckets” variable) corresponds to a specific hash partition. This specific design choice serves two key purposes: bucket-level locking rather than globally locking an entire node and efficient data management and recovery. As for the first point, fine-grained locking, each “bucket” contains its own `std::mutex` called “`bucket_mutex`”. This means that a write to bucket “0” will not block a concurrent write to bucket B of the same node. This significantly reduces lock contention compared to a coarse-grained (node-level) lock, which directly improves write throughput. Of course, this also means that writing to one bucket in one node, will not affect another write from happening to that same bucket on a different node. As for the second purpose, it is important to note that each bucket has a one-to-one correspondence to a node. This means that node “i” holds the primary data of bucket “i”, while node “j” holds the replica version of bucket “i” for node “i”. This is crucial for failure recovery because if node “0” were to die, we know that on a node that contains node “0’s” replica data, we can just look inside its bucket 0. This is a crucial optimization, where an entire bucket can be grabbed and transferred over to another node’s same bucket, rather than scanning through a massive list of key-value pairs and extracting only relevant pairs. This logic is present in the `TransferData` RPC, where it simply serializes and transmits the contents of a specific bucket ID (i) to a new node.

Finally, there is the client library, which provides multiple API, providing an abstraction to the complexity of the distributed system from a user perspective. There are three API calls which are used: `init()`, `put()`, and `get()`. `init(int id)` takes in a client id and establishes a channel to the manager and seeds the random number generator which will be used in “get”. As for `put(key, val)`, it takes in two parameters, key and val. First, it queries the manager for a set of nodes (K of them to be exact) where we can store this key-value pair in. Then it synchronously writes the data to all K nodes. A crucial design of this “write” operation is that it is an “all-or-nothing” operation; that is, the method only returns true if all writes succeed, ensuring strong consistency. Finally, the `get(key)` method queries the manager for the set of nodes that stores this key. Then, the client (not the manager), performs read load balancing by selecting a random replica node from the list to start at, to send the `Get` RPC to. It does so utilizing the `rand()` function, which was seeded in the client’s initialization. Finally, the method returns the corresponding value or an empty value if nothing is found.

With these components solidified, it is important to discuss the design principles. There are three key behaviors of the system: data partitioning (sharding), data replication, and data consistency. The system implements sharding to achieve horizontal scalability. The mechanism

which is used to achieve this is a modulo hashing algorithm defined in the `get_bucket_id()` function defined in `gstore.hpp`. Within this function, the hash of the key is taken and then modded by the number of buckets we have in our system. In code it looks like: `hash(key) % num_buckets`. The result of this is every key is mapped to a specific “bucket ID”, and since the number of buckets equals the number of nodes (N) on our system, this effectively shards the dataset across the cluster. In the unlikely event that each key maps to the same node, this would degenerate to a “data-all-in-one-node” system. However, in the average case, this modulo hashing technique prevents any single node from becoming a storage bottleneck, as is shown in the output/histogram below. This output is the result of a load balance test ran, where 100,000 inserts were performed on a system configured with 7 nodes 1 replica. Through the printed output, it is evident that the keys are distributed roughly evenly across all 7 nodes. To fully understand the “distribution” column, just note that the scale used is a “#” for every 500 keys inserted on a node. Thus, because there are ~14000 keys on each node, we expect 28 “#’s” on each row, which is exactly what we get.



Node ID	Key Count	Distribution
0	14161	#####
1	14279	#####
2	14334	#####
3	14241	#####
4	14443	#####
5	14150	#####
6	14392	#####

Figure 1. Load Balancing Histogram

To ensure high availability and fault tolerance, the system implements a chain replication strategy, based off a logical ring topology. This design ensures that the system can survive a failure of up to $K-1$ nodes without any complete data loss. To address data replication, it was decided that given a replication factor K , the data would be stored on some primary node determined by the hashing function and the next direct $K-1$ nodes. This is accomplished during the metadata lookup phase inside of the `GetNodeForKey` RPC on the manager’s side. When a client initiates a key-value write, the manager first determines the primary node using the modulo hash (noting that there is a one-to-one correspondence between nodes and buckets). It then calculates the full replica node set by walking the “ring”. As implemented in `manager.cpp`, the manager iterates from $i = 0$ to `num_reps`, where `num_reps` is equal to the replication factor, calculating each new target node as `(bucket_id + i) % number_of_nodes_on_system`. This creates a deterministic chain where the data for bucket “ i ” is stored on node “ i ”, node “ $i+1$ ”, node “ $i+2$ ”, ..., node “ $i+(K-1)$ ”. The manager then sends a response back to the client of the list of addresses (corresponding to each node), where the client is now responsible for writing to all the returned nodes, while still ensuring the strong consistency behavior.

Now addressing fault tolerance, the logic is contained within `handle_node_failure()` of `manager.cpp`. Upon the detection of a node failure, the manager executes a thorough recovery process that restores the replication factor for every single chain the dead node was a part of. This means that whether the node was the primary node for some data and the replica node for others, all data stored on this node would be replicated onto some other node(s), ensuring there is always K copies of each data in the system. The crucial understanding for the recovery algorithm stems from realizing that a single node failure affects K distinct data partitions. These include the partition for which the node was the “primary” for, and the K-1 partitions for which it served as a “replica” node. The code iterates this “K” number of times (replication factor), calculating each new “bucket_to_fix” using the “reverse-ring” logic which is as follows: “(dead_node_id - i + num_buckets) % num_buckets”. This has us iterate from the primary (current node/bucket) all the way to the K-1 bucket. Now, for each “bucket_to_fix”, the manager identifies a “backup”/“source” node which currently holds a copy of the data inside the bucket and a “target” node, which is the node we will write the new replica to, to maintain the integrity of the replication factor. The backup node is selected as the first “alive” node found beginning at the bucket’s “primary” node. The target node is selected by searching for the K-th “alive” node, again, starting from the bucket’s “primary” node. Each time the backup and target nodes are found, the manager initiates a `TransferData` RPC which copies the specific bucket’s data from the backup to the new target, ensuring we obey the “K replica” rule within the system.

Finally, the system adheres to a strong consistency model (“all-or-nothing”) with regards to write operations (“put”). A “put” operation is atomic from the client’s perspective as mentioned earlier. It does not return success until K nodes have been written to for a single “put” operation. As a result of the writes being synchronous across all K selected nodes, a “get” request (read operation) necessarily guarantees retrieval of the latest version of the requested key-value pair. This maximizes data consistency and safety and significantly simplifies “read” logic, but at the cost of increasing write latency, waiting for each node to complete a full write. In the context of systems design, this would be considered a tradeoff, albeit a seemingly necessary one.

To validate the functionality of the entire system, a client driver application called “test_app.cpp” was implemented. Here, it parses command-line arguments to execute specific tests, such as the provided `single_set_get()` function and custom “put” and “get” operations. For “put” operations, it expects the following: `./bin/test_app --put <key> --val <value>`. For “get” operations, it expects the following: `./bin/test_app --get <key>`. With these optional commands, a test user can input custom key-value pairs into the system. It is important to note that the manager and storage nodes must already be started. These test results can be found in Appendix A-D.

Now, it is crucial to discuss the design tradeoffs which are present in the current implementation of GTStore. Each of these architectural decisions were made in an attempt to balance complexity, performance, and consistency. The first tradeoff is the “write-all” data implementation which prioritizes consistency over latency. Currently, the system enforces a

“write-all” strong consistency model, where if a client performs a “put” operation, it is not considered successful or complete until the data has been written to all K replica nodes. While this is a desired behavior within distributed systems, to ensure a “get” request to any node is guaranteed to return the most up-to-date version of the data, this comes at the cost of the latency of a “put” operation. Especially because of the RPCs made, which are blocking in this case, we must wait for all writes to finish before the system can move forward. Another design tradeoff is the implementation of static modulo hashing, which prioritizes simplicity over scalability. Currently, the system uses a modulo-based hashing algorithm, mentioned previously as “ $\text{hash}(\text{key}) \% \text{num_buckets}$ ”, where “ num_buckets ” = N (number of nodes at system initialization). The key thing to note is that N is fixed at system initialization. While our current lookup/placement efficiency is $O(1)$, the problem arises if the number of nodes in our system at some points surpasses N (becomes dynamic rather than static). Once there are more nodes added after the initialized N nodes, calculating the target bucket for every key required an extremely complex data migration handling, if even realistic. Finally, there is the use of fine-grained bucket-level locking, over a coarse-grained node-level lock, which prioritizes throughput over memory efficiency and simplicity. Currently, each bucket possesses its own mutex (i.e. bucket 0, bucket 1, ...). This means that in theory we can write to multiple buckets on the same node at once, allowing for high concurrency (i.e. a write to bucket 0 doesn’t block a simultaneous write to bucket 1 on the same node). This, however, comes at the cost of increased memory overhead (where each node has N number of mutexes) and increased complexity of more locks to manage rather than being able to lock one whole node at once.

While the system successfully operates under the expected scenarios in the tests provided, it is also important to note that the design decisions discussed so far have their own limitations/restrictions. The first of which is the “partial write” problem. While the client indicates failure if a write does not occur on all K nodes (returns false), the system does not implement a rollback mechanism. This means that the writes which did occur to $<K$ nodes will stay, and the remaining are not present. This leads to an inconsistent state, where a future read may return different results or error completely. To address this, and ensure true atomicity, a rollback mechanism could be implemented. If a write to node “ i ” succeeded but that same write failed on node “ j ”, upon catching the failed write (client catches), the client can send a “delete” request to all nodes that contain this “partial write” (node “ i ”) to undo it. The only limitation with this is if a “get” request occurred before the “delete” request went through to node “ i ” meaning a dirty or nonexistent data would be read. In addition, if the client process were to unexpectedly crash before sending the “delete” request, the data may remain inconsistent within the system. Another limitation is the manager as a single point of failure. Currently, the system relies on a single centralized manager process. This means if the manager fails, the entire system falls apart; clients cannot communicate with the manager anymore, thus, cannot interact with the system entirely. To address this, the manager could be replicated as well. Finally, there is the assumption that the system initialized with N storage nodes will only ever have at most N storage nodes. If the system were able to have storage nodes come and go randomly, this would cause

`handle_node_failure()` to break and other lookup/placement logic. This is because we assume that the “number of buckets” equals the “number of nodes on our system”, and if we add $> N$ nodes, the hashing function would no longer “correctly” map key to location.

As for testing, both performance and load balancing were analyzed. The performance test performed 200,000 operations (50% read and 50% writes) on three separate system configurations: 7 nodes 1 replica, 7 nodes 3 replicas, and 7 nodes 5 replicas. The following were the results:

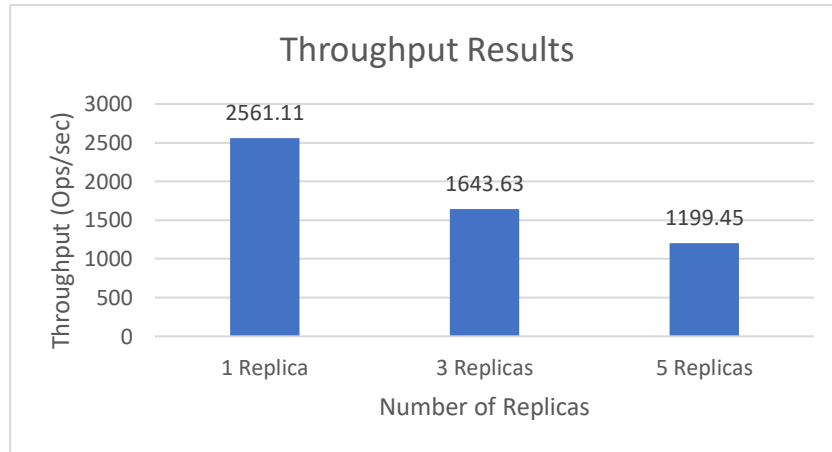


Figure 2. Performance test results

These tests demonstrate a clear correlation between the replication factor (K) and system throughput, which drops from 2561 Ops/sec, when $K=1$, to 1643 Ops/sec, when $K=3$, and then 1199 Ops/sec, when $K=5$. This behavior can be directly attributed to network overhead inherent in the “write-all” model. To explain the specific interval drops, it is important to note that they are driven by the write operations. The write operations in this system have a linear cost because a “put” request requires the client to serialize and transmit data to K distinct nodes. On the other hand, we note that read (“get”) operations do not change the throughput all that much if at all, as they always have a constant cost due to them only needing to query a single node for data. We also note that the rate of throughput degradation slows as K increases (it drops more significantly from $K=1$ to $K=3$, than it does from $K=3$ to $K=5$). This is due to the diminishing effect of the additional overhead of the “write” operations. As we add more replicas, the cost of adding these diminishes. Finally, it is important to realize the latency is bounded by the synchronous behavior of the system. The “put” function makes blocking gRPC calls to “Put”, which means it waits for an acknowledgement (i.e. via `status.ok()`) from one write before initiating the next (to another node). Consequently, the total time for a write operation degenerates to the sum of all “Put” network calls. As an aside, please note that for these performance tests, all “cout” statements were commented out as they were significantly affecting throughput. In addition, for tests 1-4, the tests were run individually, although in the `./run.sh` script they are included, the outputs may become intertwined with one another.

Appendix A

```
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key1 --val val1
> OK, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key1
> key1, val1, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key
1 --val value2
> OK, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key2 --val value3
> OK, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key3 --val value4
> OK, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
1
> key1, value2, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
2
> key2, value3, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
3
> key3, value4, 0.0.0.0:50052
```

Figure 3. Test 1 Results

Appendix B

```
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key1 --val value1
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key1
> key1, value1, 0.0.0.0:50056
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key
1 --val value2
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key
2 --val value3
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key3 --val value4
> OK, 0.0.0.0:50055 0.0.0.0:50056 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
1
> key1, value2, 0.0.0.0:50056
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
1
> key1, value2, 0.0.0.0:50056
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key1
> key1, value2, 0.0.0.0:50055
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
2
> key2, value3, 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
3
> key3, value4, 0.0.0.0:50055
```

Figure 4. Test 2 Results

Appendix C

```
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key1 --val value1
> OK, 0.0.0.0:50053 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key1 --val newvalue1
> OK, 0.0.0.0:50053 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key2 --val value2
> OK, 0.0.0.0:50053 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key3 --val value3
> OK, 0.0.0.0:50052 0.0.0.0:50053
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key4 --val value4
> OK, 0.0.0.0:50054 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key5 --val value5
> OK, 0.0.0.0:50054 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key6 --val value6
> OK, 0.0.0.0:50052 0.0.0.0:50053
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --put key7 --val value7
> OK, 0.0.0.0:50053 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key1
> key1, newvalue1, 0.0.0.0:50052
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
2
> key2, value2, 0.0.0.0:50054
jsamp35@JOSHUA-HP:/mnt/c/Users/JOSHUA SAMPSON/Project4/gtstore$ ./bin/test_app --get key
7
> key7, value7, 0.0.0.0:50052
```

Figure 5. Test 3 Results

Appendix D

```
Now let's populate our data
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
> OK, 0.0.0.0:50056 0.0.0.0:50057 0.0.0.0:50058
> OK, 0.0.0.0:50056 0.0.0.0:50057 0.0.0.0:50058
> OK, 0.0.0.0:50052 0.0.0.0:50053 0.0.0.0:50054
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50056 0.0.0.0:50057 0.0.0.0:50058
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50056 0.0.0.0:50057 0.0.0.0:50058
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50058 0.0.0.0:50052 0.0.0.0:50053
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50058 0.0.0.0:50052 0.0.0.0:50053
> OK, 0.0.0.0:50052 0.0.0.0:50053 0.0.0.0:50054
> OK, 0.0.0.0:50052 0.0.0.0:50053 0.0.0.0:50054
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
> OK, 0.0.0.0:50055 0.0.0.0:50056 0.0.0.0:50057
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50058 0.0.0.0:50052 0.0.0.0:50053
Now let's overwrite some keys are told by project test description
> OK, 0.0.0.0:50054 0.0.0.0:50055 0.0.0.0:50056
> OK, 0.0.0.0:50053 0.0.0.0:50054 0.0.0.0:50055
> OK, 0.0.0.0:50058 0.0.0.0:50052 0.0.0.0:50053
```

```
Now kill 2 nodes

./test4.sh: line 32: 148769 Killed                  ./bin/storage --port $CURR_PORT
Recovered lost data!!! Placed into NODE ID: 4
Recovered lost data!!! Placed into NODE ID: 3
Recovered lost data!!! Placed into NODE ID: 2
Recovered lost data!!! Placed into NODE ID: 0
Recovered lost data!!! Placed into NODE ID: 6
Recovered lost data!!! Placed into NODE ID: 5
./test4.sh: line 33: 148793 Killed                  ./bin/storage --port $CURR_PORT
NOTE: expected key-value pairs are as follows...
<key1, val1>, <key2, va2>, <key3, val3>, ..., <key20, val20>
WITH THREE EXCEPTIONS:
<key1, UPDATED_VAL_1>, <key10, UPDATED_VAL_10>, <key20, UPDATED_VAL_20>

> key1, UPDATED_VAL_1, 0.0.0.0:50054
> key10, UPDATED_VAL_10, 0.0.0.0:50054
> key20, UPDATED_VAL_20, 0.0.0.0:50058
> key3, val3, 0.0.0.0:50057
> key9, val9, 0.0.0.0:50057
> key17, val17, 0.0.0.0:50054
TEST 4 DONE!!!
```

Figure 6. Test 4 Results

These are partial outputs from the entire script. The script executed was test4.sh.

Appendix E

```
Inside GTStoreManager::init()  
Number of Nodes (N) = 3 Replication Factor (K) = 2  
We are listening on 0.0.0.0:50051  
□
```

Figure 7. Manager Initialization

Appendix F

```
Inside GTStoreManager::init()
Number of Nodes (N) = 3 Replication Factor (K) = 2
We are listening on 0.0.0.0:50051
Manager has responded to Register request, created new node ID: 0 Address: 0.0.0.0:50052
Manager has responded to Register request, created new node ID: 1 Address: 0.0.0.0:50053
Manager has responded to Register request, created new node ID: 2 Address: 0.0.0.0:50054
█
```

```
Inside GTStoreStorage::init()
Successfully registered node ID: 0
Number of buckets: 3
█
```

```
Inside GTStoreStorage::init()
Successfully registered node ID: 1
Number of buckets: 3
█
```

```
Inside GTStoreStorage::init()
Successfully registered node ID: 2
Number of buckets: 3
█
```

Figure 8. Storage Node Creations

Appendix G

```
Inside GTStoreManager::init()
Number of Nodes (N) = 3 Replication Factor (K) = 2
We are listening on 0.0.0.0:50051
Manager has responded to Register request, created new node ID: 0 Address: 0.0.0.0:50052
Manager has responded to Register request, created new node ID: 1 Address: 0.0.0.0:50053
Manager has responded to Register request, created new node ID: 2 Address: 0.0.0.0:50054
Recovered lost data!!! Placed into NODE ID: 0
Recovered lost data!!! Placed into NODE ID: 2
█
```

Figure 9. Handling Node Failure (Manager), here Node 1 on 50053 was killed